INTERCOLLEGE INDUSTRIAL TALK

Agentic AI in production

The hard part is the system around the model.

Course materials: bit.ly/prod-ai-github

LinkedIn: linkedin.com/in/jfomhover

OPENING / PERSPECTIVE

Jean-François Omhover

AI Product & Platform Leader · 25 years across research, product, and enterprise AI

2005–2016

Arts et Métiers ParisTech

Associate Professor

ML research · product design
research management and leadership

2017–2026

Microsoft

Applied Science → Product

ML in compliance · privacy-preserving ML
Azure ML · Microsoft Foundry

2026–present

BNP Paribas

AI Tech Lead

Enterprise AI · architecture
trusted systems

Research → product and platforms → enterprise AI

© J.F. Omhover · 2026 bit.ly/prod-ai-github

OPENING / CLASS PHILOSOPHY

This class is a steering prompt

Implementation details are abundant. Direction is harder to find.

ABUNDANT ONLINE

How to call a model?

How to build an agent?

How to connect a tool?

AI can help fill in the details.

THIS CLASS

What changes in an enterprise setting?

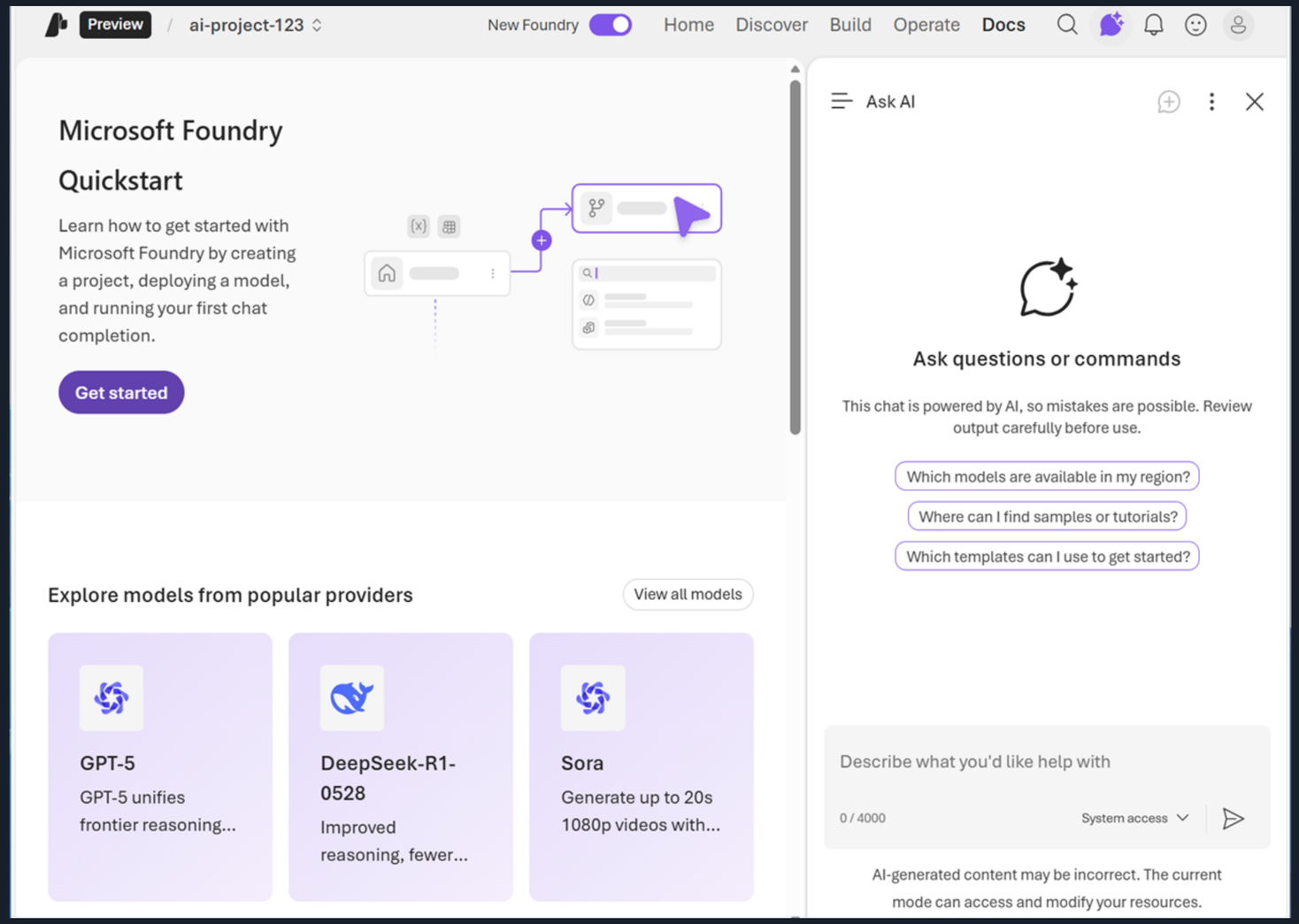
Which concerns are easy to overlook?

Which concerns become blockers later?

What is critical to succeed?

The hour is for direction, not implementation coverage.

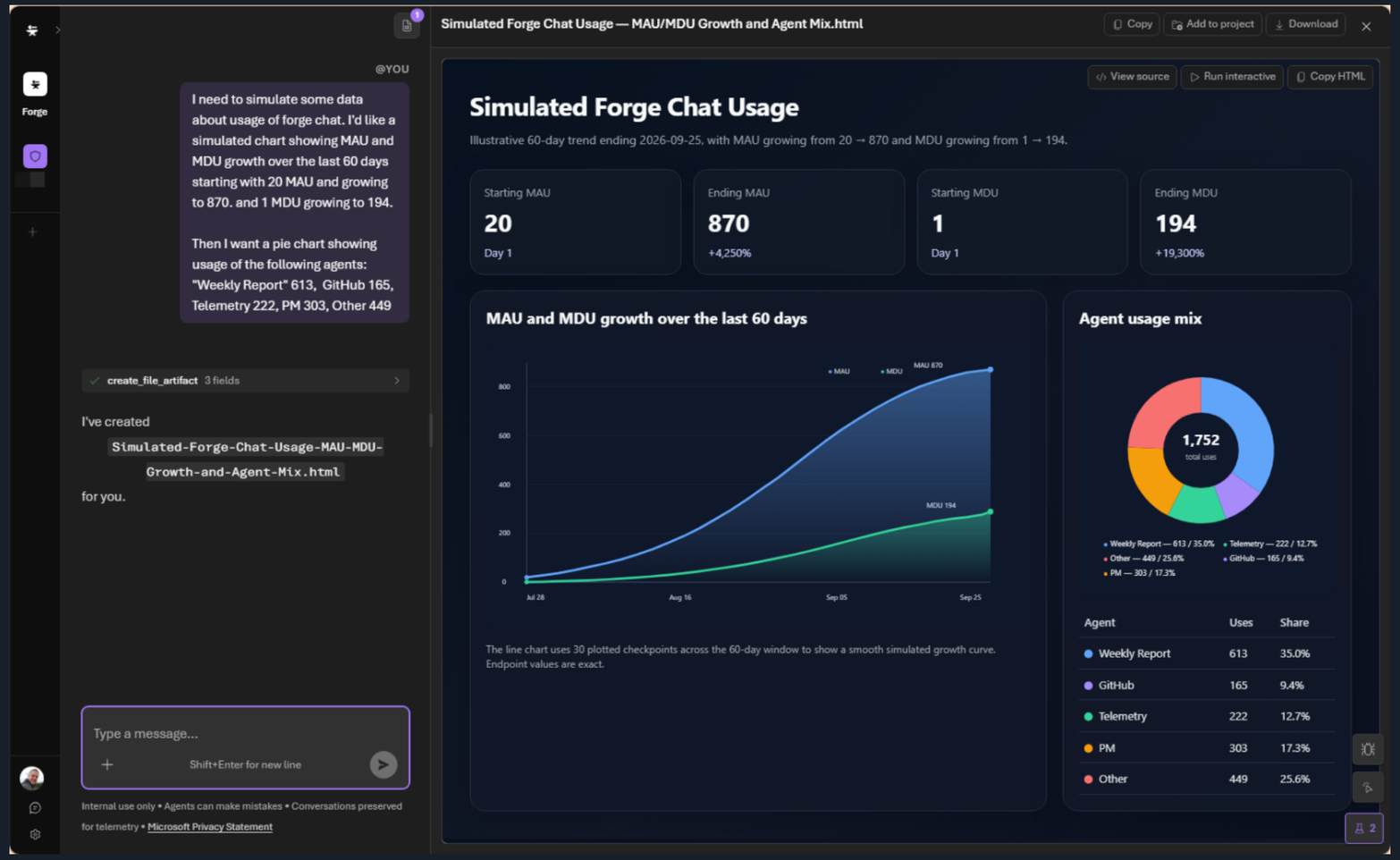
AskAI: an action-enabled agent in a studio interface



Critical enterprise concerns

- data residency and compliance
- user consent and action execution
- cross-functional delivery and ownership

Forge: people and agents in shared work



Critical enterprise concerns

- agentic fit for process management
- cross-tenant identity management
- action policy validation

Forge Chat, Chris Harris, 2026

Internal-use example, not a product announcement.

OPENING / CLASS MAP

Four questions for enterprise agentic AI

A review lens for the system around the model.

01 / AGENT FIT

What is an agent good at?

Where should simpler software take over?

02 / USER-DATA MANAGEMENT

What data does it handle?

What is captured, stored, and exposed?

03 / IDENTITY AND DELEGATION

Whose authority is in use?

What can the system do?

04 / POLICY ENFORCEMENT

How is authority enforced?

What stops an unsafe action?

© J.F. Omhover · 2026 bit.ly/prod-ai-github

An agent is not a general-purpose employee in software form.

In this section, we will:

01 describe the bounded envelope of a reliable agent;

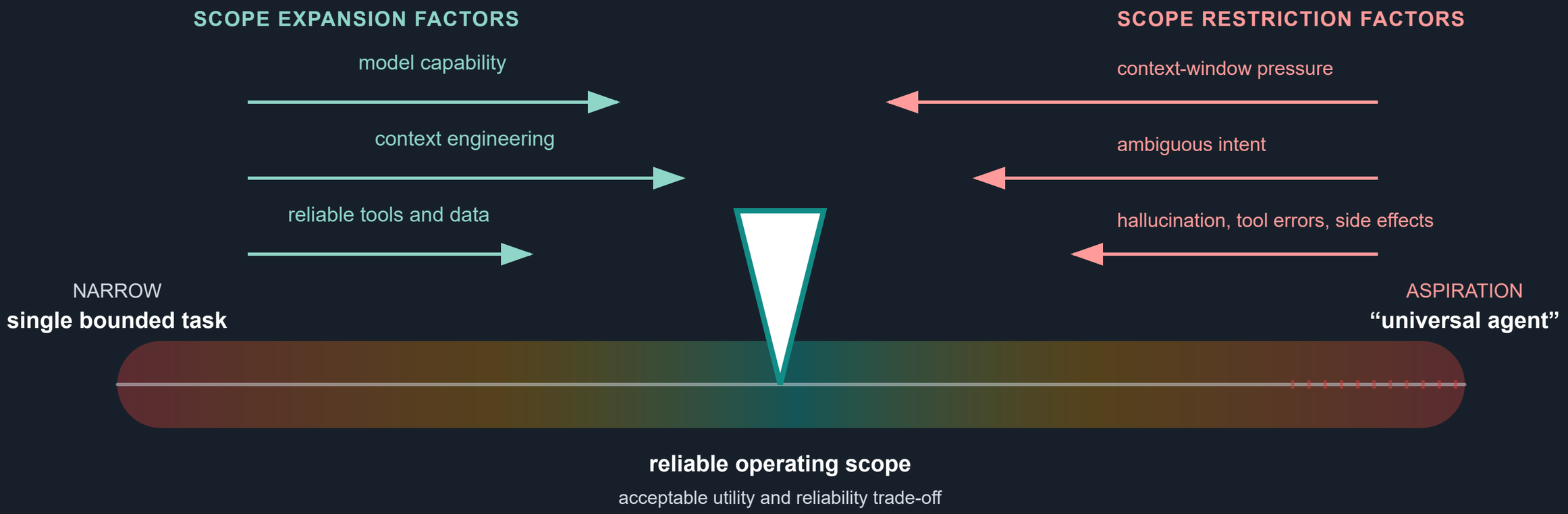
02 see why broad scope makes evaluation harder;

03 separate adaptive work from deterministic automation.

What is an agent actually good at, and where should simpler software take over?

Every agent has a practical scope limit

Some forces push scope larger; others push reliable scope smaller.



Capability raises the ceiling; controls and evidence make more of it safe to use.

Broad scope multiplies the evaluation surface

A plausible response does not prove that the right action was pursued.

NARROW GOAL

Known inputs

Defined success and failure

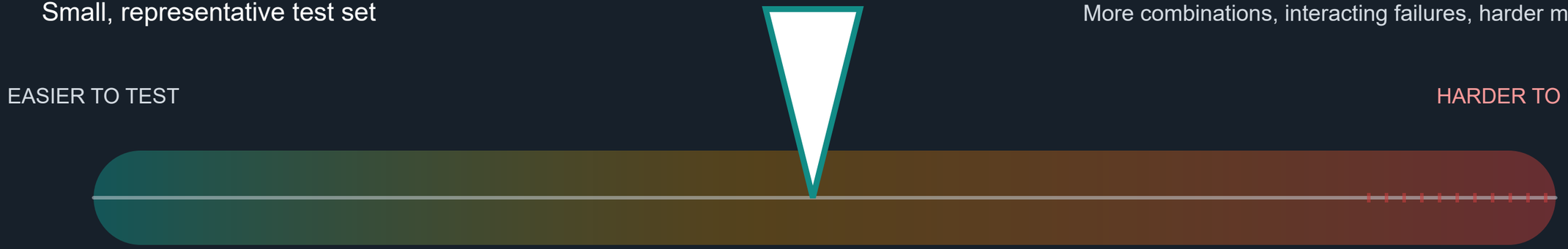
Small, representative test set

MANY-GOAL AGENT

intent × context × retrieval

× tool state × permission × outcome

More combinations, interacting failures, harder monitoring



testable operating scope

the cursor moves only when evidence supports it

Narrow goals make alignment and failure boundaries inspectable.

Repetition belongs in a script

700 features. 15 self-reported gates. Which work belongs to an agent?

USE CASE

700 features shipping

Each feature has 15 release gates:

security · documentation · marketing ops

Evidence is self-reported across several DevOps systems.

Goal

identify risk and nudge product owners.

TWO DESIGN OPTIONS

OPTION 1 / AGENT DOES EVERYTHING

scan 700 → find systems → track work items → apply policies → report issues → ping owners in Teams

Broad scope · expensive repetition · hard to stabilize and align

OPTION 2 / SCRIPT VALIDATES, AGENT NUDGES

collect work items → one JSON → deterministic policies → report → agent sends a clear message and deadline to the owner

Repeatable validation · bounded agent · useful human interaction

Keep the compliance result deterministic. Leave the nudging to the agent.

The script is cheaper and more reliable to run daily across 700 features.

© J.F. Omhover · 2026 bit.ly/prod-ai-github

Can a coding agent keep its constraints in context?

A final thought experiment about long-horizon coding work.

SCENARIO

A coding agent has skills for architecture, secure coding, and excellent unit tests. The session grows toward test N.

OPEN QUESTION

When test N is designed, how do you know the right constraints are still active?

© J.F. Omhover · 2026

An agent creates a data flow, not just a response.

In this section, we will:

01 separate user data from personal data;

02 trace data through the workflow;

03 build a small, useful inventory.

What data does the system handle, where does it go, and what controls does each step require?

© J.F. Omhover · 2026

User data is not just PII

Under GDPR, personal data is a defined legal term. User data is the wider engineering view.

User data

Information supplied, touched, or produced through an interaction.

- prompts and files
- retrieved context
- tool arguments and results
- model responses
- memory, approvals, and traces

Personal data (GDPR)

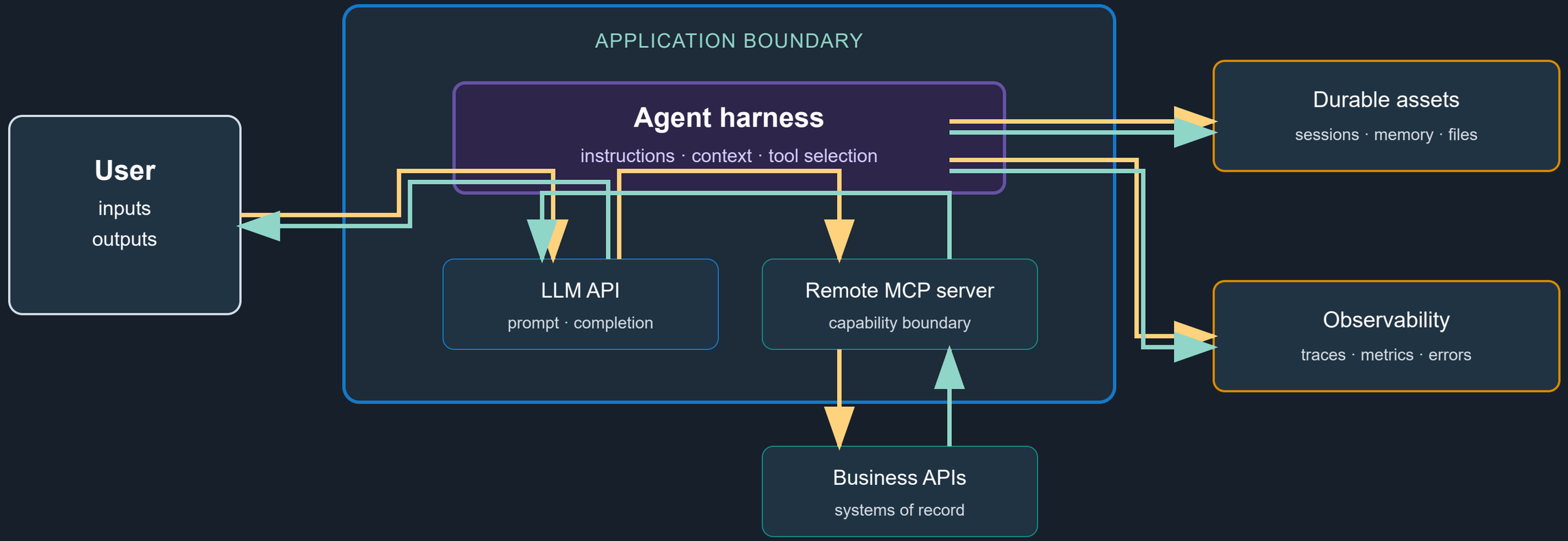
Information relating to an identified or identifiable natural person.

- health information
- customer records
- identifiers linked to a person
- evaluations or inferences
- traces about an identifiable user

Do not inventory only names, phone numbers, and government IDs.

Where user data flows in an agent system

A minimal architecture schema for the data and trust boundaries.



Add the data paths after agreeing on the component boundaries.

user data flow tool or system result

© J.F. Omhover · 2026 bit.ly/prod-ai-github

Start with a small inventory

Seven practical questions are enough to expose the first gaps.

INVENTORY FIELD	QUESTION
Data	What is captured or transmitted?
Classification	What kind of data is it?
Purpose	Why does the system need it?
Storage	Which system holds or processes it?
Access	Who can read or change it?
Retention	When is it deleted?
Transfer	Which boundary does it cross?

Example record

Data	User prompt
Classification	User content; may be personal
Purpose	Continue session context
Storage	Session storage
Access	User + session service
Retention	30 days, or when user deletes it*
Transfer	Model provider request

*Example policy, not a universal rule.

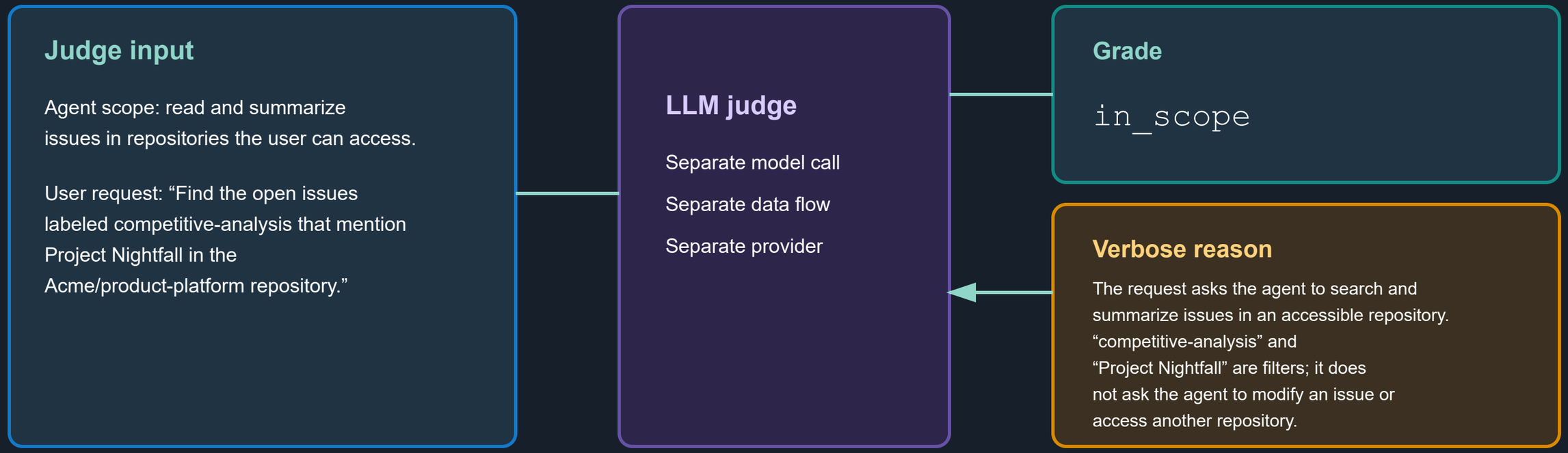
Use the inventory to audit the data flow.

Regulations · user agreements · policies · contracts

© J.F. Omhover · 2026 bit.ly/prod-ai-github

⚠ An LLM judge creates another processing flow

Internal evaluation can still process personal or confidential data.



A verbose reason may repeat the request, secret product name, labels, or repository details.

Treat the reason as an inventory item: classify it and apply the same data controls.

When the lab drifts from production

A final thought experiment about privacy-preserving evaluation drift.

SCENARIO

Lab: curated offline evaluation dataset.

Production: unpredictable user prompts, tools, and traces.

Raw data is restricted; aggregate signals and LLM judges remain.

OPEN QUESTION

How do you detect drift between the lab evaluation set and production behavior without inspecting raw prompts?

© J.F. Omhover · 2026 bit.ly/prod-ai-github

An agent should borrow authority, not become the authority.

In this section, we will:

01 distinguish identity, authorization, and delegation;

02 trace authority through the remote GitHub MCP boundary;

03 limit each action by least privilege and approval.

Whose authority is the agent using, and how can it act without receiving more privilege than the user has?

© J.F. Omhover · 2026

Why privileged access fails

A service credential can give an agent authority that does not belong to the user.

Broad service credential

Many users and repositories

Revocation may not follow the user

Audit sees the service, not the person

Large blast radius

Delegated user authority

User access remains attributable

The resource checks user permissions

Capabilities can be narrowed per task

Smaller action boundary

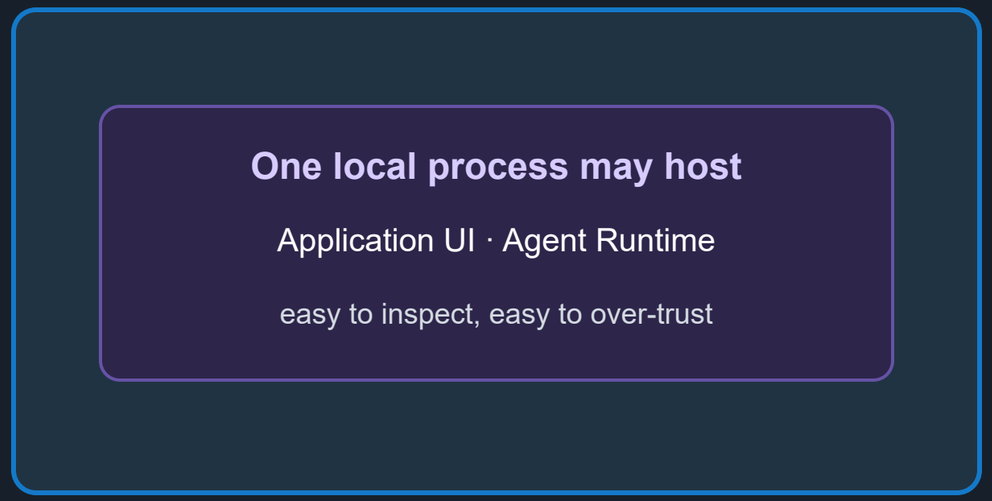
Delegation limits who the agent represents; least privilege limits what it can do.

© J.F. Omhover · 2026

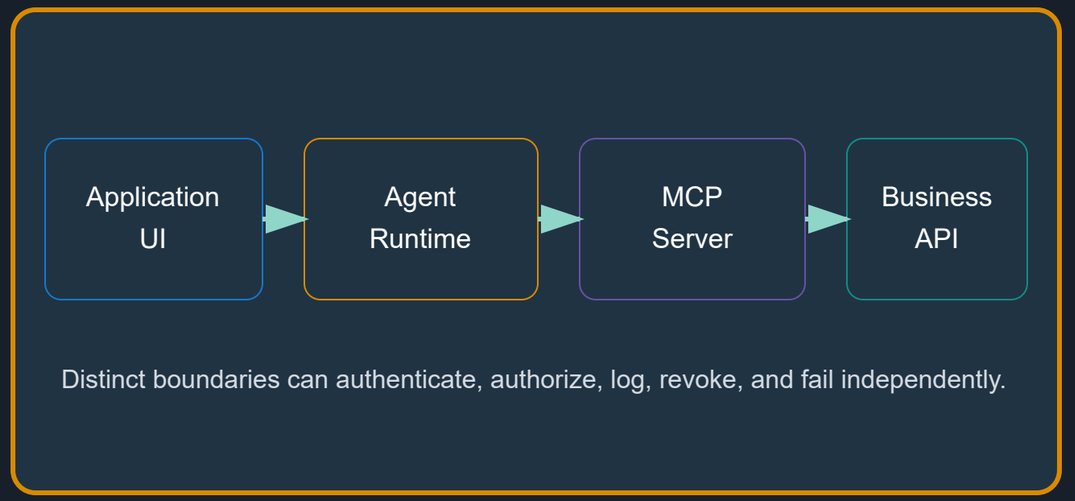
Prototypes collapse identities; deployment separates them

One local process may host several roles; deployment makes trust boundaries explicit.

LOCAL PROTOTYPE



DEPLOYED SYSTEM



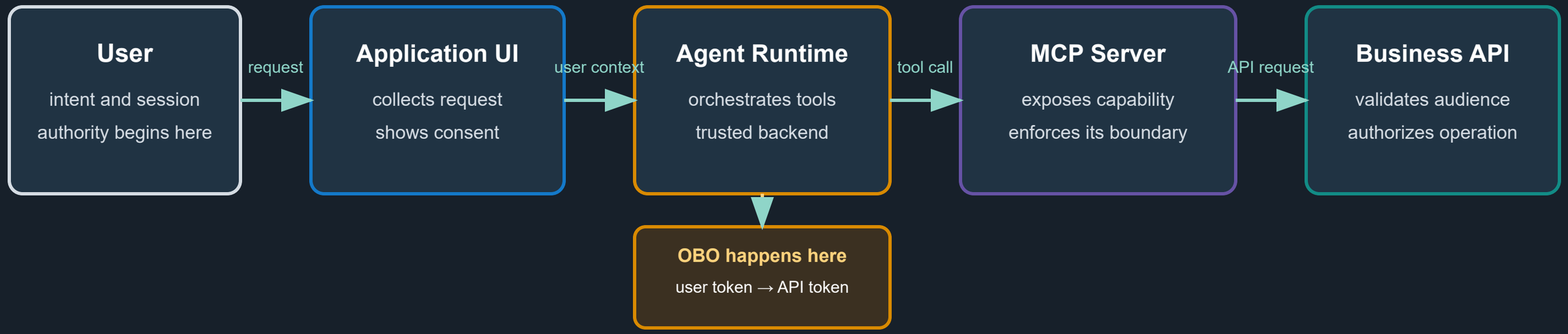
Why separate identities?

User identity represents a human; system identity represents a workload or service.

An identity provider authenticates principals and issues credentials; APIs still authorize requests.

Where on-behalf-of happens

Keep the UI, runtime, MCP server, and business API as distinct boundaries.



OBO: the runtime exchanges a user token for a token targeting a downstream API.

Tokens are credentials: validate issuer, audience, expiry, and permissions.

The model receives data and tool results, never the token.

A token carries claims for an intended audience; JWT is one format, and opaque tokens also exist.

Every component must align; every condition must be met

Every boundary must permit the same concrete operation.

PROPOSED OPERATION

Create a comment on issue #42 in Acme/product-platform.

APPLICATION + PROVIDER CONDITIONS

User access
can access repo and issue #42

Application scope
includes issue-comment API

MCP capability
write tool exposed with parameters

Task policy
operation allowed for this purpose

WORKFLOW + DOWNSTREAM GATES

Approval
record is valid for this request

GitHub authorization
downstream permits the operation

all required conditions permit the proposed operation

Approval and downstream authorization must also succeed; otherwise, do not execute.

© J.F. Omhover · 2026 bit.ly/prod-ai-github

One agent, many users

A final thought experiment about identity and cross-user exposure.

SCENARIO

Alice and Bob use the same deployed agent.

Each has private session memory and separate system access.

One shared runtime handles both conversations.

OPEN QUESTION

How do you guarantee user session isolation from one another?

Policy comes before action, not after the tool call.

In this section, we will:

01 test delegated authority against the task requested;

02 inspect the harness seam before a tool runs;

03 separate inputs, decisions, and application outcomes.

How is authority enforced before an agent can act?

Delegation is not enough

A valid user credential does not resolve ambiguous intent.

Scenario: GitHub conversation

USER

“Clean up the old release issue in Acme/product-platform.”

AGENT

“I found #1842, Project Nightfall release checklist. I’ll close it now.”

Ambiguous target + destructive action

Why delegation is not enough

Delegation answers: who is acting?

It does not answer:

- Is #1842 the intended issue?
- Did the user intend to close it?
- Is approval required for this action?

Task policy narrows delegated authority.

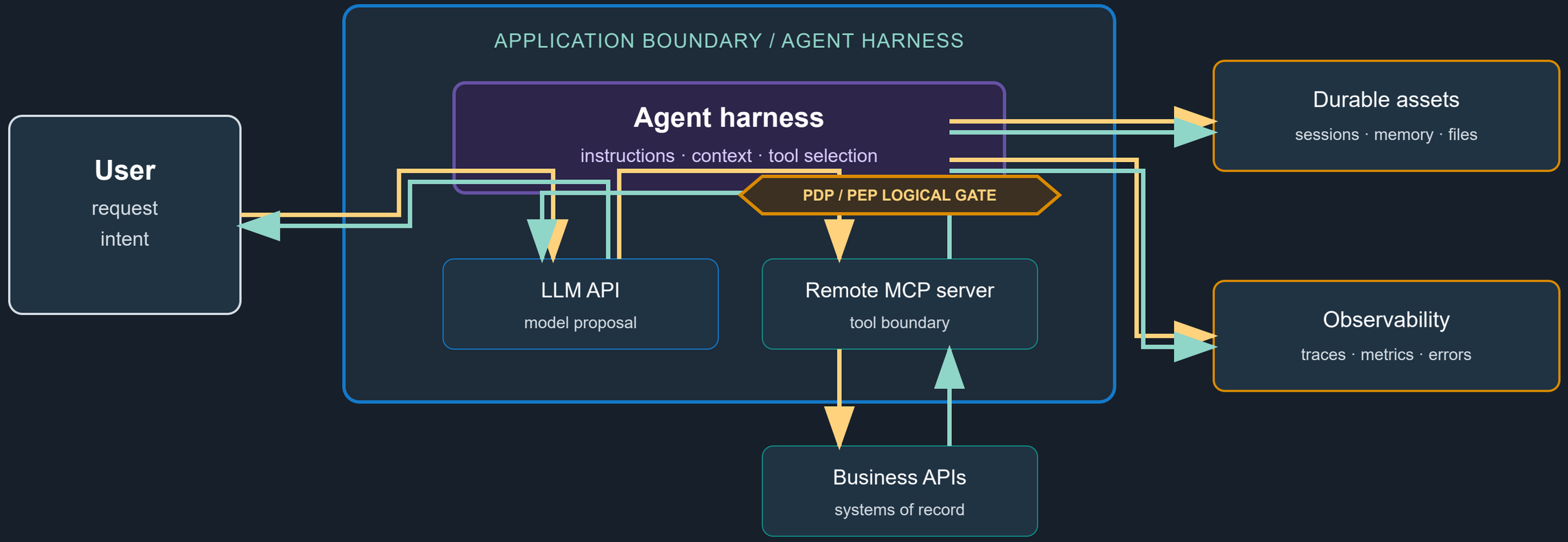
We can’t solve this by only writing better instructions.

Require explicit intent, a known target, and approval where the action is consequential.

© J.F. Omhover · 2026

Where the harness meets policy validation

The harness checks each proposed operation against explicit policy before the tool runs.



The model proposes. Policy decides. The harness enforces. The resource server authorizes.

A policy request needs four things

NIST ABAC makes the decision inputs explicit.

Subject

Who is asking?

user=demo-user

agent=backlog-agent

Object

What is affected?

repository=example/repo

issue=42

Operation

What should happen?

github.issue.comment.create

Environment

What else matters?

user_can_write=true

approval=false

There is no universal policy-request schema. These are the typed inputs for this example.

Policy result versus workflow outcome

The policy engine and the application do not use the same vocabulary.

Policy results

Permit

Deny

Indeterminate

NotApplicable

Application workflow

Execute

Require approval*

Retry or show an error

Stop

*Application state, not a universal PDP result.

If approval is required but no valid record exists, stop before the GitHub call.

How do you govern a bash tool?

A final thought experiment about policy boundaries.

SCENARIO

The agent can call a bash tool that executes just any script.

OPEN QUESTION

How do you create a policy applicable to it?

Script · arguments · files · network · identity · approval · effects

Where would you enforce it, and what evidence would prove it works?

© J.F. Omhover · 2026 bit.ly/prod-ai-github

CLOSING / NEXT QUESTIONS

Thank you for your attention

The production questions continue beyond this class.

FOUR PRODUCTION QUESTIONS

01 / AGENT FIT

What is an agent actually good at,
and where should simpler software take over?

02 / USER DATA

What data does the system capture, create,
store, transmit, and expose?

03 / IDENTITY

Whose authority is the agent using,
and what can it do?

04 / POLICY

How is authority enforced before an agent can act?

WORTH INVESTIGATING NEXT

DURABLE AGENTS

How can an agent be interrupted,
checkpointed, and resumed safely?

ZERO-TRUST ARCHITECTURES

How should trust boundaries and
per-request verification shape the system?

GUARDRAILS

Where should guardrails live,
and what happens when they fail?

Continue the conversation

linkedin.com/in/jfomhover

© J.F. Omhover · 2026 bit.ly/prod-ai-github